Phased Testbed Build Plan

Reactive SDN for Securing ICS Environments — Project CARS

Siemens S7-1200/1500 in-loop · Ryu + Open vSwitch · Modbus TCP + S7comm · July 2026

0. Safety & isolation — read before powering anything

You are putting a real PLC in-loop, so the testbed must be treated as a physical process. Three non-negotiables:

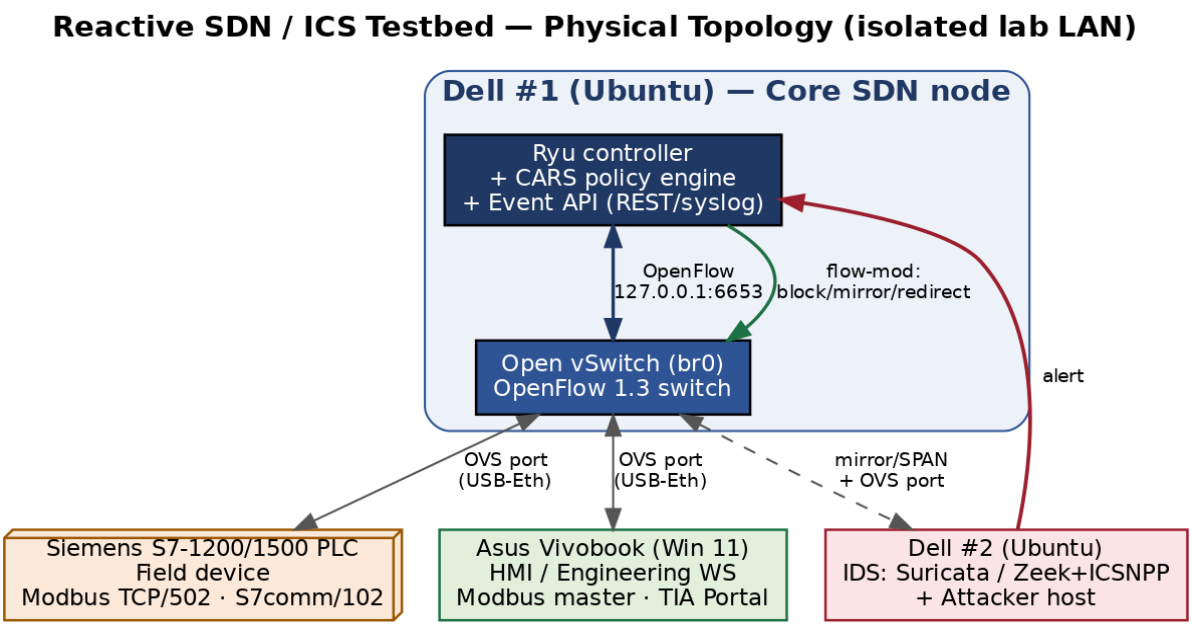
- **Air-gap the lab.** Build on a dedicated, isolated LAN with no bridge to home/corporate networks or the internet. The S7-1200/1500 has known unauthenticated-write exposure; it must never be reachable from outside the bench.
- **Use a harmless process.** Drive only safe I/O — LEDs, a small 24 V lamp/relay, or a simulated tank in the PLC program. No motors, heaters, or anything that can cause harm if a flow rule or attack misfires.
- **Fail-safe defaults.** The controller's default action on uncertainty is mirror/allow, never block, so a bug in the response logic cannot itself trip the process (this is also the core CARS principle).

1. Hardware inventory & role assignment

Device	Role	Notes
Dell #1 (Ubuntu)	Core SDN node	Ryu controller + Open vSwitch (br0) + CARS policy/Event API. The heart of the testbed. Needs 2–3 Ethernet ports → add USB-to-Ethernet adapters.
Dell #2 (Ubuntu)	IDS + attacker	Suricata and/or Zeek+ICSNPP as the external event source; also the attacker host for scenarios. Later: digital-twin host.
Asus Vivobook (Win 11)	HMI / engineering WS	SCADA HMI + Modbus master polling the PLC; runs TIA Portal to program the S7-1200/1500.
Siemens S7-1200/1500	Field device (crown jewel)	Real PLC. Enable Modbus TCP server (TCP/502) for experiments; S7comm (TCP/102) stays on for realism/attacks. Wired into an OVS port.
MacBook Air M1	Excluded / spare	ARM makes x86 SDN/Mininet tooling painful. Use for writing/notes only.

2. Network topology

All endpoints attach to Open vSwitch (br0) on Dell #1, so the controller mediates every flow. With limited built-in NICs, give Dell #1 extra physical ports via USB-to-Ethernet adapters and add each as an OVS port. The IDS receives a copy of traffic through an OVS mirror (SPAN) port; its alerts drive the reactive loop.



Reactive loop (green/red in the diagram): IDS detects an anomaly → sends an alert to the Event API on Dell #1 → the CARS policy engine picks a criticality-aware action → Ryu pushes a flow-mod (block / mirror / redirect) to OVS. The physical PLC and HMI keep talking through OVS the whole time.

3. Protocol strategy

Primary experimental protocol: Modbus TCP (port 502). Enable a Modbus TCP server block on the S7-1200/1500 and use a Modbus master on the Asus as the HMI. Modbus is easy to allowlist and deep-inspect, and it lines up directly with your local literature (Ndonda & Sadre's P4 Modbus allowlist; Goldenberg & Wool's Modbus/TCP model). **Secondary / realism: S7comm (port 102).** Keep S7comm enabled so you can run realistic Siemens-specific attacks (unauthorized start/stop, block download) and detect them with Zeek's ICSNPP s7comm analyzer. Design CARS to be protocol-agnostic at the policy layer so both map onto the same block/mirror/redirect actions.

4. Software stack

Layer	Tool	Host
SDN controller	Ryu (Python OpenFlow 1.3 apps)	Dell #1
Software switch	Open vSwitch (br0)	Dell #1
Reactive logic	CARS app + Event API (Flask/REST or syslog listener)	Dell #1
IDS / detection	Suricata (signatures) + Zeek + ICSNPP (Modbus/S7comm parsers)	Dell #2
HMI / master	Modbus master (e.g. pymodbus/QModMaster) + TIA Portal	Asus
Process logic	Ladder/SCL program with Modbus server + safe I/O	Siemens PLC
Attacker	pymodbus, Snap7, nmap, scapy, hping3	Dell #2
Capture / analysis	Wireshark, tcpdump, ovs-ofctl	Dell #1/#2

5. Phased build plan

Six phases. Each lists its goal, key steps, the exit criterion (how you know it's done), and the papers from the master reading list to read alongside it. Phases A–B are the minimum viable testbed; C–D deliver the CARS contribution; E–F are evaluation and stretch.

Phase A — Bench prep & physical connectivity

- Install Ubuntu, Ryu, Open vSwitch, Wireshark on Dell #1; create bridge br0; add physical ports (built-in + USB-Ethernet) as OVS ports.
- Program the S7-1200/1500 in TIA Portal: a simple safe process (e.g., simulated tank level or traffic-light) with a Modbus TCP server block; assign a static lab IP.
- Cable PLC, Asus (HMI) and Dell #2 (IDS/attacker) into OVS ports; set the whole bench on one isolated subnet.
- **Exit criterion:** HMI on the Asus can read/write PLC registers over Modbus, with traffic physically passing through OVS (confirmed in Wireshark).
- **Read alongside:** MiniCPS; “Oops I Did It Again” ICS testbed survey; Goldenberg & Wool (Modbus modeling).

Phase B — SDN baseline

- Run a Ryu learning-switch app (simple_switch_13); confirm OVS is controller-managed (ovs-vsctl show, ovs-ofctl dump-flows).
- Verify the HMI ↔ PLC Modbus loop still works under controller-installed flows; measure baseline latency/jitter (important: control-loop deadlines).
- **Exit criterion:** You can add/remove a flow rule from Ryu and see the effect on PLC ↔ HMI traffic in real time.
- **Read alongside:** Piedrahita (IEEE Software + the Computer Networks extended version); Ndonda & Sadre P4 two-level IDS.

Phase C — Detection integration (external event source)

- Deploy Suricata and Zeek+ICSNPP on Dell #2; feed them via an OVS mirror/SPAN port so detection is passive and never in the control path.
- Write/enable Modbus and S7comm rules (unauthorized write, function-code abuse, scanning); emit alerts as structured events (EVE JSON / syslog).
- Stand up the Event API on Dell #1 that ingests those alerts in a normalized schema (source, device, flow, severity, suggested action).
- **Exit criterion:** An attacker action on Dell #2 produces an alert that arrives at the Event API within a bounded, logged time.
- **Read alongside:** Enabling Dynamic Network Access Control (anomaly IDS + SDN); DIDEROT (DNP3 IDPS pattern); DPI for software-defined industrial networks.

Phase D — CARS reactive response engine (the contribution)

- Connect the Event API to a Ryu CARS app that can install block, mirror, and redirect flow rules on demand.
- **Criticality model:** tag each device/flow (e.g., PLC ↔ HMI control loop = critical; unknown host = untrusted). Asset identity feeds the "unseen device" decision.
- **Safety guard:** on a critical flow, never hard-block — mirror or redirect for inspection instead; only block untrusted/unseen devices. Validate rule changes with a policy check before install.
- **Exit criterion:** An unseen device that alerts is blocked automatically, while an alert on the critical PLC loop is redirected for inspection — with the physical process never interrupted.
- **Read alongside:** Hammar (Optimal Security Response); SoK ICS Asset Discovery; A Policy Checker Approach for Secure Industrial SDN.

Phase E — Attack scenarios & evaluation

- Run a scenario suite from Dell #2: network scan/recon, unauthorized Modbus/S7 writes, false-data injection, and a Modbus/DoS flood.
- **Metrics:** detection-to-mitigation latency; process-safety preservation (did the loop stay in bounds?); false-positive rate; controller/switch load. Log everything for reproducibility.
- Benchmark methodology against SWaT/WADI/HAI conventions so results are comparable to the literature.
- **Exit criterion:** A repeatable results table quantifying reaction latency and safety preservation per scenario — the core evaluation for the dissertation.
- **Read alongside:** Samanis (Bristol MTD thesis — method/eval template); Securing ICS networks: Automated Traffic Control + MTD; iTrust SWaT/WADI/HAI.

Phase F — Stretch: line-rate, deception, and twin validation

- P4 data-plane path (bmv2/software P4 switch) for a line-rate Modbus allowlist that offloads the fast path from the controller.
- Redirect-to-honeypot response (extend CARS with an on-demand decoy) and/or a lightweight MiniCPS digital twin to pre-validate a disruptive action before applying it.
- **Read alongside:** P4Control; Programmable Data Planes for OT; Reactive cyber deception; TwinSec-IDS; Digital Twin-Enhanced Incident Response.

6. Start here — the minimal first milestone

Do not try to build all six phases before testing anything. Your first concrete goal is a trivial end-to-end reactive loop that proves the plumbing:

- PLC running a safe Modbus process, HMI polling it, all traffic through OVS (Phase A).
- Ryu managing OVS (Phase B).
- A hand-crafted trigger (even a manual curl to the Event API, before the IDS is wired) that makes Ryu install one block/redirect rule and visibly change the traffic.

Once that loop works end-to-end, everything else is incremental. Getting this minimal loop running will teach you more about the stack than any amount of additional reading — which is exactly why the reading and the build run in parallel.

7. Key risks & mitigations

Risk	Impact	Mitigation
Not enough physical NICs on Dell #1	Can't attach all endpoints to OVS	2–3 USB-to-Ethernet adapters; or VLANs on a cheap managed switch.
Response logic disrupts the live PLC loop	Process/safety incident	Fail-safe default = mirror/allow; safety guard blocks only untrusted devices; policy check before install.
S7comm hard to deep-inspect	Weak detection on native protocol	Use Modbus TCP as primary experimental protocol; Zeek ICSNPP for S7comm signatures.
Controller/OVS latency breaks control-loop timing	PLC faults / instability	Measure baseline jitter (Phase B); pre-install rules for the critical loop; keep IDS off the control path (mirror only).
Real PLC exposed beyond the bench	Serious security risk	Strict air-gap; static lab subnet; no internet/corporate bridge.

This plan maps directly onto MASTER_READING_LIST.md: Phases A–D correspond to reading Phases 0–3, and each build phase above names the exact papers to read alongside it. Tick reading and build items together.